

SQL JOINS

Simple Guide for Beginners

Why JOINS Feel Hard - and How to Make Them Easy

Table	What it means
EMPLOYEES	People who work in the company
DEPARTMENTS	Teams / groups in the company
JOBS	Job titles and salary ranges
LOCATIONS	Cities where offices are located

All examples use Oracle HR schema (the same tables you already know!)

Prepared for: Tank54 Group

Part 1: Why Do JOINS Feel Hard?

Many students say: "JOINS are the hardest part of SQL." But why? Let us look at the 5 main reasons. When you understand the problem, the solution becomes much easier.

1

You must think about TWO tables at the same time

Before JOINS, you work with one table. `SELECT * FROM employees`. Easy. But a JOIN asks you to look at two tables and find connections between them. Your brain must hold Table A + Table B + the rule to connect them, all at once. This is like trying to read two books at the same time. It takes practice.

2

Which table is LEFT and which is RIGHT?

When you write: `SELECT ... FROM employees e JOIN departments d ...` Which one is LEFT? Which one is RIGHT? This confuses many people. The answer is simple: The first table (FROM) is always LEFT. The second table (JOIN) is always RIGHT. Remember this rule!

3

Some rows disappear and you do not know why

You write a JOIN. You expect 15 rows. But the result has only 12. Where did 3 rows go? This happens because some rows in one table do not have a match in the other table. INNER JOIN removes unmatched rows. This is the #1 surprise for beginners. We will fix this with visual examples below.

4

Too many types: INNER, LEFT, RIGHT, FULL, SELF...

There are many types of JOINS. When you see all of them on one page, it looks like a wall of text. But the good news is: you only need to learn 4 types for 95% of your work. The rest are rare. We will focus on the 4 most important ones in this guide.

5

Venn diagrams are beautiful but abstract

Most guides show circles (Venn diagrams). They look nice but they do not show you what happens to each ROW. You need to see actual data: which row matches, which row gets NULL, which row disappears. That is why this guide uses real data examples, not just circles.

Part 2: JOINS = A Simple Matching Game

Think of a JOIN as a matching game. You have two lists. You must find matching items from both lists.

Real-life example:

List 1 (LEFT): People and their favorite colors

Person	Favorite Color
Ali	RED
Ayse	BLUE
Memed	GREEN

List 2 (RIGHT): Colors and their prices

Color	Price (\$)
RED	10
BLUE	20
YELLOW	30

Now let us match!

1

Ali likes RED. RED is in List 2. Price = \$10. MATCH!

2

Ayse likes BLUE. BLUE is in List 2. Price = \$20. MATCH!

3

Memed likes GREEN. GREEN is NOT in List 2. NO MATCH!

4

YELLOW has price \$30. But nobody likes YELLOW. NO MATCH!

This is exactly what a JOIN does! It connects rows from two tables based on a common column. The common column here is Color.

INNER JOIN Result (only matches):

Person	Favorite Color	Price (\$)
Ali	RED	10
Ayse	BLUE	20

Notice: Memed is missing (no GREEN in List 2). YELLOW is missing (nobody likes it). This is how INNER JOIN works: only matching rows survive.

Part 3: INNER JOIN (The Most Common)

INNER JOIN = Only keep rows that match in BOTH tables. If a row has no match, it is REMOVED from the result. This is the most used JOIN type. About 80% of all JOINS are INNER JOINS.

Think of it as:

"I want employees who work in a department. If an employee has no department, do not show them. If a department has no employees, do not show it."

SQL Syntax:

```
SELECT e.first_name, e.last_name, d.department_name  
FROM employees e  
INNER JOIN departments d  
ON e.department_id = d.department_id;
```

Key parts explained:

- 1 FROM employees e -> LEFT table (first table)
- 2 INNER JOIN departments d -> RIGHT table (second table)
- 3 ON e.department_id = d.department_id -> MATCHING RULE
- 4 e and d are aliases (short names for tables)

The ON clause is the most important part! It tells SQL which column to use for matching. Without it, SQL does not know how to connect the two tables. Think of it as the "matching rule" from our color game example.

INNER JOIN: What Happens to Each Row?

Let us trace each row step by step. This is how SQL thinks internally.

EMPLOYEES (LEFT table):

employee_id	first_name	department_id
100	Steven	90
101	Neena	90
102	Lex	NULL
178	Kimberely	10

DEPARTMENTS (RIGHT table):

department_id	department_name
10	Administration
90	Executive
20	Marketing

Row-by-row matching:

Row 1: Steven (dept_id = 90)

Does dept_id 90 exist in DEPARTMENTS? YES (Executive).

Result: Steven | Executive (MATCH - kept)

Row 2: Neena (dept_id = 90)

Does dept_id 90 exist in DEPARTMENTS? YES (Executive).

Result: Neena | Executive (MATCH - kept)

Row 3: Lex (dept_id = NULL)

Can you match NULL with anything? NO. NULL means "no value".

Result: Lex is REMOVED from result!

Row 4: Kimberely (dept_id = 10)

Does dept_id 10 exist in DEPARTMENTS? YES (Administration).

Result: Kimberely | Administration (MATCH - kept)

What about Marketing (dept_id = 20)?

No employee has department_id = 20.

Result: Marketing department is REMOVED from result!

Final INNER JOIN Result:

first_name	department_name
Steven	Executive

first_name	department_name
Neena	Executive
Kimberely	Administration

Key idea: INNER JOIN keeps ONLY rows that match in BOTH tables. Lex is gone (NULL department). Marketing is gone (no employees). Only matched rows survive!

Part 4: LEFT (OUTER) JOIN

LEFT JOIN = Keep ALL rows from the LEFT table. If there is no match in the right table, fill with NULL. This is useful when you want to see ALL employees, even those without a department.

Think of it as:

"Show me ALL employees. If someone has no department, just write NULL for the department name. Do NOT remove anyone from the list."

SQL Syntax:

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

LEFT vs INNER: The only difference is one word! Change "INNER JOIN" to "LEFT JOIN" and now ALL left table rows are kept.

Same data - now with LEFT JOIN:

first_name	department_name
Steven	Executive
Neena	Executive
Lex	NULL
Kimberely	Administration

Look at Lex: He is BACK in the result! His department_name is NULL because he has no department_id. But LEFT JOIN does not remove him. Marketing is still missing (it is in the RIGHT table and has no employees).

When to use LEFT JOIN?

- You want to find employees with NO department: WHERE d.department_name IS NULL
- You want to see ALL customers, even those without orders
- You want a complete list from the main table, with optional extra info
- You are doing a report and cannot miss any rows from the primary table

Part 5: RIGHT (OUTER) JOIN

RIGHT JOIN = Keep ALL rows from the RIGHT table. If there is no match in the left table, fill with NULL. This is the opposite of LEFT JOIN.

Think of it as:

"Show me ALL departments. If a department has no employees, just write NULL for the employee name. Do NOT remove any department from the list."

SQL Syntax:

```
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

Result with RIGHT JOIN:

first_name	department_name
Steven	Executive
Neena	Executive
Kimberely	Administration
NULL	Marketing

Now Marketing is BACK! It has NULL for first_name because no employee works there. But RIGHT JOIN keeps it because Marketing is in the RIGHT table (departments).

Lex is missing again because he is in the LEFT table and has no match. RIGHT JOIN protects the RIGHT table, not the LEFT.

Pro tip: You can swap the tables and use LEFT JOIN instead of RIGHT JOIN. Both give the same result. Most developers prefer LEFT JOIN because it is easier to read (the main table comes first).

Part 6: FULL OUTER JOIN

FULL OUTER JOIN = Keep ALL rows from BOTH tables. If there is no match on one side, fill with NULL. This is like LEFT JOIN + RIGHT JOIN combined!

Think of it as:

"Show me EVERYTHING. All employees AND all departments. Put NULL wherever there is no match. Do not remove any row from either table."

SQL Syntax:

```
SELECT e.first_name, d.department_name  
FROM employees e  
FULL OUTER JOIN departments d  
ON e.department_id = d.department_id;
```

Result with FULL OUTER JOIN:

first_name	department_name
Steven	Executive
Neena	Executive
Lex	NULL
Kimberely	Administration
NULL	Marketing

EVERYONE is here! Lex has NULL department (no match). Marketing has NULL employee (no match). No row is removed. This is the most complete JOIN.

Part 7: All 4 JOINS Compared

Here is a summary of all 4 JOIN types. Compare them side by side.

JOIN Type	LEFT table rows	RIGHT table rows	Best for
INNER JOIN	Only matches	Only matches	When you need only connected data
LEFT JOIN	ALL kept	Only matches	Complete list from main table
RIGHT JOIN	Only matches	ALL kept	Complete list from second table
FULL OUTER JOIN	ALL kept	ALL kept	Complete data from both tables

Visual Summary:

INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL OUTER JOIN
Only matching rows from both	All from LEFT + matches from RIGHT	Matches from LEFT + All from RIGHT	All rows from both tables

Part 8: Common Mistakes and How to Avoid Them

Mistake 1: Forgetting the ON clause

Some students write: `SELECT * FROM employees e JOIN departments d`
This is WRONG! You need the ON clause to tell SQL how to match.

Correct: `... JOIN departments d ON e.department_id = d.department_id`

Mistake 2: Using WHERE instead of ON

Some students write: `JOIN departments d WHERE e.department_id = d.department_id`
This sometimes works but it is not the correct way. Always use ON for the join condition.

Mistake 3: Wrong column in ON clause

Some students join on the wrong column. For example, joining employees to departments on `employee_id`. That does not make sense! You must join on the column that CONNECTS the two tables. Look for columns with the same name (like `department_id`).

Mistake 4: Not using table aliases

Without aliases, you must write the full table name every time:

`employees.first_name, departments.department_name ...`

With aliases, it is much shorter: `e.first_name, d.department_name`

Always use aliases. Your code will be cleaner and easier to read.

Mistake 5: Confusing LEFT and RIGHT

Remember this simple rule: The FROM table is always LEFT. The JOIN table is always RIGHT. So if you want to keep all employees, employees must be in the FROM position, and you use LEFT JOIN.

Part 9: Practice Examples (Oracle HR)

Now let us see real examples with the Oracle HR schema. These examples use the tables you already know from previous lessons.

Example 1: Employee names with their department names

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id
ORDER BY d.department_name;
```

This shows every employee and the department they work in. Employees with no department are not shown (INNER JOIN removes them).

Example 2: Find employees with NO department

```
SELECT e.first_name, e.last_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

This is a very common pattern! LEFT JOIN keeps all employees. Then WHERE d.department_id IS NULL filters to show only those who did not match any department. Think of it as: "Show me employees who have no department."

Example 3: Department names with their manager names

```
SELECT d.department_name, e.first_name, e.last_name
FROM departments d
INNER JOIN employees e ON d.manager_id = e.employee_id
ORDER BY d.department_name;
```

Here we join departments with employees using manager_id. This shows who manages each department. The connection column is manager_id (in departments) = employee_id (in employees). The ON columns do not always have the same name!

Example 4: Employee salary with their job title

```
SELECT e.first_name, e.last_name, j.job_title, e.salary
FROM employees e
INNER JOIN jobs j ON e.job_id = j.job_id
WHERE e.salary > 10000
ORDER BY e.salary DESC;
```

This joins employees with jobs to show the job title (not just job_id) for employees who earn more than 10000. The JOIN adds meaningful information (job title) that does not exist in the

employees table alone.

Example 5: Employees with their city name

```
SELECT e.first_name, e.last_name, l.city  
FROM employees e  
INNER JOIN departments d ON e.department_id = d.department_id  
INNER JOIN locations l ON d.location_id = l.location_id  
ORDER BY l.city;
```

This uses two JOINS! We connect: employees -> departments -> locations. You can add more JOINS to connect more tables. Each JOIN adds one more table to your query. The key is to find the connecting column between each pair of tables.

Part 10: Quick Reference Card

Cut out this page and keep it on your desk. Use it when you write JOINS.

What you want	Which JOIN	SQL Template
Only matching rows from both tables	INNER JOIN	FROM A INNER JOIN B ON A.id = B.id
All rows from first table	LEFT JOIN	FROM A LEFT JOIN B ON A.id = B.id
All rows from second table	RIGHT JOIN	FROM A RIGHT JOIN B ON A.id = B.id
All rows from both tables	FULL OUTER JOIN	FROM A FULL OUTER JOIN B ON A.id = B.id
Find rows with no match (left)	LEFT JOIN + IS NULL	FROM A LEFT JOIN B ON... WHERE B.id IS NULL
Connect 3 tables	Multiple JOINS	FROM A JOIN B ON A.id = B.a_id JOIN C ON B.id = C.b_id

Golden Rules:

1. Rule 1: FROM table is always the LEFT table. JOIN table is always the RIGHT table.
2. Rule 2: The ON clause tells SQL which columns to match. Never forget it!
3. Rule 3: INNER JOIN removes unmatched rows from BOTH tables.
4. Rule 4: LEFT JOIN keeps all left rows. Missing matches get NULL.
5. Rule 5: NULL never equals NULL. So NULL values cannot match anything.
6. Rule 6: Use table aliases (e, d, j, l) to make your code shorter and cleaner.
7. Rule 7: Start with INNER JOIN. Only use LEFT/RIGHT/FULL when you specifically need it.

Part 11: Exercises

Try these exercises on your own. Use Oracle HR schema. Check your answers by running the queries in your database.

Easy (BEGINNER level):

Exercise 1: Write a query to show first_name and department_name for ALL employees. Use INNER JOIN.

Exercise 2: Write a query to show first_name and job_title for ALL employees. Join employees with jobs.

Exercise 3: Write a query to show department_name and city for ALL departments. Join departments with locations.

Medium (PRACTICE level):

Exercise 4: Find employees who have NO department. Show their first_name and last_name. (Hint: LEFT JOIN + IS NULL)

Exercise 5: Show the manager name for each department. Join departments with employees. Show department_name, manager's first_name and last_name.

Exercise 6: Show first_name, last_name, salary, and min_salary from the jobs table. Find employees whose salary is less than the min_salary for their job.

Exercise 7: Show city and the number of departments in each city. Join locations with departments. Use COUNT(*).

Hard (CHALLENGE level):

Exercise 8: Show first_name, last_name, department_name, city, and country_name. You need to join 4 tables: employees, departments, locations, countries.

Exercise 9: Find departments that have no employees. Show department_name.

Exercise 10: For each job title, show the number of employees and the average salary. Use JOIN + GROUP BY. Order by average salary from high to low.

Remember: JOINS become easy with practice. Do not try to memorize everything. Write queries every day. Start with INNER JOIN. When you feel comfortable, try LEFT JOIN. Step by step, you will master all JOIN types. Good luck!